FLARE: Faithful Logic-Aided Reasoning and Exploration

Erik Arakelyan[†15] **Pasquale Minervini**[23] **Pat Verga**[4] **Patrick Lewis**[4] **Isabelle Augenstein**[1]
[1]University of Copenhagen [2]University of Edinburgh
[3]Miniml.AI [4]Cohere [5]NVIDIA
earakelyan@nvidia.com augenstein@di.ku.dk p.minervini@ed.ac.uk
{pat, patrick}@cohere.com

arXiv:2410.11900v5 [cs.AI] 19 Sep 2025

Abstract

Modern Question Answering (QA) and Reasoning approaches with Large Language Models (LLMs) commonly use Chain-of-Thought (CoT) prompting but struggle with generating outputs faithful to their intermediate reasoning chains. While neuro-symbolic methods like Faithful CoT (F-CoT) offer higher faithfulness through external solvers, they require code-specialized models and struggle with ambiguous tasks. We introduce **F**aithful **L**ogic-**A**ided **R**easoning and **E**xploration (**FLARE**), which uses LLMs to plan solutions, formalize queries into logic programs, and simulate code execution through multi-hop search without external solvers. Our method achieves SOTA results on **7** out of **9** diverse reasoning benchmarks and 3 out of 3 logic inference benchmarks while enabling measurement of reasoning faithfulness. We demonstrate that model faithfulness correlates with performance and that successful reasoning traces show an 18.1% increase in unique emergent facts, 8.6% higher overlap between code-defined and execution-trace relations, and 3.6% reduction in unused relations.

1 Introduction

Complex Reasoning in natural Question Answering (QA) tasks requires exploring a problem space with formalized facts, relations, commonsense knowledge and logical implications. In line with this, LLMs have been enhanced with CoT (Wei et al., 2022) prompting, which supplements the QA process by generating intermediate reasoning chains given a set of in-context examples (Brown et al., 2020a), as shown in fig. 1. This allowed for advancement in commonsense (Madaan et al., 2022), symbolic (Wang et al., 2022; Sprague et al., 2024) and mathematical (Jie et al., 2023) reasoning. Although CoT allows for a problem exploration in natural language steps, such an approach has been shown to cause performance degradation for reasoning tasks involving multi-step planning (Valmeekam et al., 2022; Suzgun et al., 2023), problem exploration (Yao et al., 2022), and arithmetic tasks (Hendrycks et al., 2021a; Madaan and Yazdanbakhsh, 2022a). These discrepancies arise as CoT suffers from a limited ability to decompose, search, verify and backtrack using intermediate rationale chains (Yao et al., 2022), cascading hallucinations and errors (Ling et al., 2023) and that natural language might not be an optimal representation for describing the reasoning process (Li et al., 2024). Simultaneously, LLM output has been shown to be unfaithful and inconsistent w.r.t. the intermediate CoT rationale (Jacovi et al., 2024; Lanham et al., 2023b; Turpin et al., 2023).

To mitigate the problem of CoT faithfulness and allow for more robust reasoning during QA, Lyu et al. (2023, Faithful CoT) and Logic-LM (Pan et al., 2023) suggested generating code which is further executed using an external symbolic solver. Producing and executing code enables the generation of outputs guided by external solvers, leveraging search with backtracking to explore the problem space effectively. However, strict translations of natural language queries into code, such as *auto-formalisation* (Szegedy, 2020; Wang et al., 2018), is a non-trivial task involving direct inference of implicit commonsense and domain-specific knowledge and the ability to align abstract and informal concepts directly to constrained formal definitions for further execution (Wu et al., 2022). An example query, *"Do all parts of the aloe vera plant taste good?"*, is challenging to formalize or address with a strict algorithmic solution, as it requires interpretative, deductive and context-dependent reasoning, referred to as soft or fuzzy reasoning. Using external solvers makes such fuzzy reasoning impossible and requires consistently generating syntactically correct executable code. While some LLMs have coding capabilities stemming from their pretraining

[†]Corresponding author.

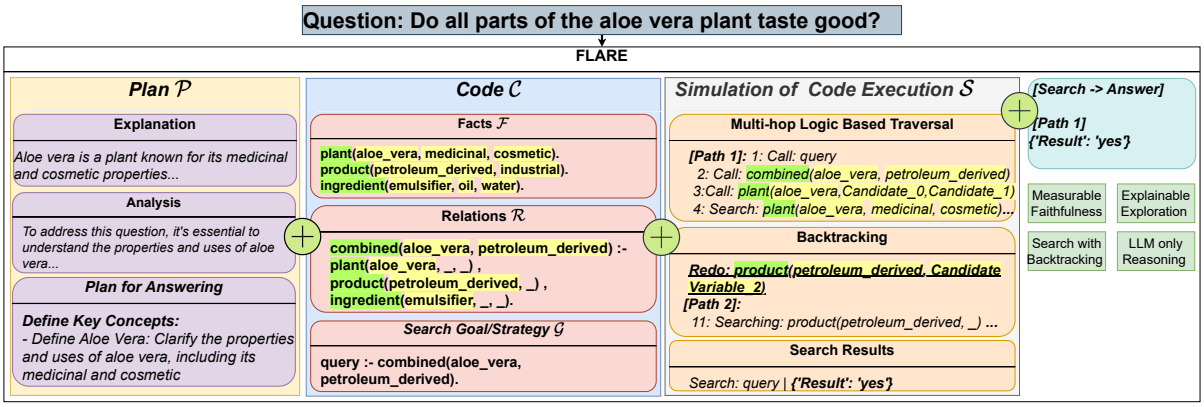

Figure 1: A depiction of the *plan*, *code* and simulated *search* in FLARE. Each module is generated separately and iteratively, allowing us to obtain the final answer. The green and yellow highlighted text shows the overlap between the facts and the relations between the code and the simulated search.

(Jiang et al., 2024; Aryabumi et al., 2024), relative code consistency is more probable with models explicitly trained for coding (Chen et al., 2021).

To overcome these problems, we propose Faithful Logic-Aided Reasoning and Exploration (FLARE), an interpretable method that allows for planning, fuzzy reasoning, and traversing the problem space with backtracking, exact task decomposition, and measuring faithfulness. In FLARE, given a natural language query, we prompt an LLM to sequentially generate a *plan* that includes an analysis and the logical steps necessary for formalising and answering the question, a logic programming (Wielemaker et al., 2012) *code* that allows formalising the query into a set of facts, relations and their composition forming the space for exploring that query and the *search*, which is an LLM-generated code execution simulation. An illustration of FLARE can be seen in fig. 1. This work focuses on models that **have not** been explicitly trained on CoT on other reasoning traces, as these models have been shown to struggle with generalisation towards differing reasoning paradigms (Chen et al., 2024), consistency in intermediate reasoning steps (Wang et al., 2025) and instruction following (Zhang et al., 2025). In our framework, the generated code must not be consistently executable by an external solver, allowing for the soft-formalisation of natural language. Although we see that even generalist LLMs are able to produce executable code in $\geq 50\%$ of cases. FLARE allows us to measure the faithfulness of the outcome w.r.t. the simulated code execution by directly comparing the search paths produced by the external solver to that LLM generation. This comparison also allows for pinpointing model hallucinations and inconsistencies. We systematically study the effectiveness of our method using 4 general-purpose LLMs of varying scales across 9 diverse QA and 3 logical inference benchmarks, covering Math World Problems, Multi-hop QA, Relation inference, deductive and analytical reasoning and show that our method achieves state-of-the-art results in 7 out of 9 QA datasets and 2 out of 3 logic datasets in comparison to CoT, F-CoT and Logic-LM. We also show that the method is competitive for models tuned for coding, with an average overall increase of 16% over F-Cot and 10% over CoT. Our key contributions are the following: **(i)** We introduce FLARE, a novel paradigm for logic-aided and interpretable formalisation and search over the problem space in QA and logic reasoning tasks. **(ii)** We perform a systematic evaluation across 9 QA and 3 logical inference benchmarks and 4 models of varying scales, showing the advantages of using FLARE for QA in a few-shot setup over prior approaches. **(iii)** The modularity of FLARE allows defining a simple ingrained method for measuring model faithfulness, which is further shown to be strongly correlated with performance. **(iv)** We further show that using FLARE allows us to interpretably and rigorously detect hallucinations along with sub-optimal and inconsistent reasoning patterns.

2 Related Work

Reasoning in Natural Language

Reasoning in Natural Language Few-shot prompting (Brown et al., 2020b) improves LLM reasoning, and extensions like Chain-of-Thought (CoT)(Wei et al., 2022), "think step by step"(Kojima et al., 2022), and Least-to-

Most (Zhou et al., 2023) explicitly decompose problems into intermediate steps. Despite their promise, these methods exhibit arithmetic errors (Lewkowycz et al., 2022; Hendrycks et al., 2021b) and logical inconsistencies (Madaan and Yazdanbakhsh, 2022b). Planning-based variants introduce a separate plan–execute loop (Yao et al., 2023b; Wang et al., 2023a). The *plan* stage in FLARE draws on these ideas but focuses on generating a natural-language strategy for later formalisation into code.

Reasoning with Search Recent work augments LLM reasoning by explicitly searching the problem space. Self-consistency decoding (Wang et al., 2023b) samples multiple chains of thought and selects the majority answer, while Tree-of-Thoughts (ToT; Yao et al., 2023a) performs tree-structured exploration with LLM-evaluated states. Later methods adapt classical search—DFS, BFS (Besta et al., 2024), A*(Lehnert et al., 2024), and hybrids(Gandhi et al., 2024)—via direct tuning, imitation learning (Yang et al., 2022), or few-shot prompting (Zhang et al., 2024). So far, evaluations focus on toy puzzle and algorithmic domains such as the 24 Game, Countdown, Sorting, mazes, and Sokoban (Yang et al., 2022; Wikipedia, 2024; Besta et al., 2024; Lehnert et al., 2024). Although the *search* module of FLARE shares this multi-path exploration spirit, it targets more general tasks and yields interpretable multi-hop reasoning via simulated code execution.

Reasoning with Formalisation Another research direction explores formalising natural language queries into code (Gao et al., 2023; Li et al., 2024) or pseudo-code (Chae et al., 2024; Gandhi et al., 2024). This enables translating queries into strict structures, delegating reasoning and search to deterministic solvers such as Python (Chen et al., 2023), PDDL (Lyu et al., 2023; Liu et al., 2023), or DataLog (Lyu et al., 2023). Models can synthesize programs (Austin et al., 2021; Nijkamp et al., 2023) and benefit from code in numerical and algorithmic reasoning (Chen et al., 2023; Gao et al., 2023), yet their use for general QA remains underexplored. This is due to the challenge of translating natural language into strictly executable code (Wu et al., 2022), the syntactic rigidity of underrepresented programming languages during pre-training (Liu et al., 2024), and the need for models explicitly tuned for coding (Chen et al., 2021). Additionally, relying on external solvers restricts soft reasoning over commonsense knowledge and implications. In FLARE, we formalise queries as logic programs in Prolog during the *code* generation step but do not require executability or external solvers at inference. This allows LLMs to simulate code execution via soft reasoning over logic-based traversals—similar to Prolog—while avoiding the need for code-specific tuning.

Reasoning Faithfulness An explanation is considered *faithful* if it explicitly and accurately describes the reasoning process of the model during inference (Gilpin et al., 2018; Jacovi and Goldberg, 2020). In the context of prompting techniques such as CoT, we are interested in the faithfulness of the intermediate reasoning chains towards the final output. Faithful intermediate reasoning chains should not just look *plausible* (Herman, 2017) but have exact reflections of the problem exploration and reasoning used to arrive at the final answer. Natural language reasoning chains prevalent in CoT and similar methods are shown to be unfaithful, either masking the reasoning biases (Turpin et al., 2023) of the model or outright ignoring the intermediate reasoning (Lanham et al., 2023a). In FLARE, we introduce a method to seamlessly measure the faithfulness of the final outcome w.r.t. completed search.

3 Methodology

3.1 LLM-Simulated Search

FLARE comprises three modules for generating a *plan*, *code* and simulated *search* for answering a natural language query $\mathcal{Q} = \{T_1^{\mathcal{Q}} \dots T_{|\mathcal{Q}|}^{\mathcal{Q}}\}$, where each $T_i^{\mathcal{Q}}$ is a token in the query $\mathcal{Q}$.

Generating A Plan For each query $\mathcal{Q}$, given an LLM $\mathcal{M}$, we initially use instructions $\mathcal{I}^{\mathcal{P}}$ to prompt it to generate a *plan* $\mathcal{P}$, which should be comprised of task explanation, analysis and a plan for further formalising the query. An example of this can be seen in the *plan* section in fig. 1. We use in-context few shot examples $\mathcal{E}_{\mathcal{P}}$ of such *plan* generations for obtaining the final plan:

$$\mathcal{P}_i \sim p_{\mathcal{M}}(T_i^{\mathcal{P}} \mid T_{:i-1}^{\mathcal{P}}, \mathcal{E}_{\mathcal{P}}, \mathcal{Q}, \mathcal{I}^{\mathcal{P}}), \qquad (1)$$

where $\mathcal{P}_i$ and $T_i^{\mathcal{P}}$ is the i-th token in the generated *plan* $\mathcal{P}$ and $p_{\mathcal{M}}$ is the probability of the next token over the vocabulary obtained from model $\mathcal{M}$.

Generating Code After generating the *plan*, we use instructions $\mathcal{I}^{\mathcal{C}}$ to prompt the LLM $\mathcal{M}$ to generate a Prolog code $\mathcal{C}$, an example of which can

	Math Word Problems					Multi-hop QA			Relation
Method	GSM8K	SVAMP	MultiArith	ASDiv	AQuA	StrategyQA	Date	Sport	CLUTRR
Llama-3.1-8B$_{\text{FLARE}}$	72.7	**86.0**	**96.3**	**83.1**	**62.9**	**70.2**	**59.3**	76.6	36.8
Llama-3.1-8B$_{\text{F-CoT}}$	*0*	*0*	*0*	*0*	*12.2*	53.2	*0*	*0*	*32*
Llama-3.1-8B$_{\text{CoT}}$	**85.2**	82.4	91.6	79.1	51.6	*43.5*	74.1	**89.4**	**45.7**
CmDR$_{\text{FLARE}}$	**52.4**	**74.0**	**84.5**	**72.2**	**43.7**	**67.0**	**52.3**	**78.9**	29.1
CmDR$_{\text{F-CoT}}$	*0*	*0*	*0*	*0*	*0*	59.7	*0*	*0*	*8.6*
CmDR$_{\text{CoT}}$	46.5	57.3	83.1	37.2	28.3	*21.3*	47.4	55.2	**29.5**
CmDR+$_{\text{FLARE}}$	**71.4**	**83.5**	**90.4**	**81.3**	**55.9**	**70.8**	61.8	**77.7**	**41.0**
CmDR+$_{\text{F-CoT}}$	*0*	*0*	*0*	*0*	*15.4*	57.6	*0*	*0*	*35.3*
CmDR+$_{\text{CoT}}$	48.7	81.1	86.6	44.6	44.1	*48.4*	**79.1**	62.6	42.5
GPT-3.5$_{\text{FLARE}}$	**82.1**	82.7	**98.3**	**85.4**	55.1	**65.5**	**82.4**	85.6	**49.8**
GPT-3.5$_{\text{F-CoT}}$	*75.8*	**83.0**	*95.3*	81.7	*53.5*	*51.5*	73.5	*52.3*	12.1
GPT-3.5$_{\text{CoT}}$	79.8	*82.4*	98.2	*75.8*	**59.4**	51.7	*69.9*	**95.8**	*4.3*

Table 1: The following table shows the performance of each of the tested models given a technique for reasoning. Each **bold**, underlined, and *italicised* element highlights the best, second best and worst technique per specific model. The overall best method per dataset is highlighted in green.

be seen in fig. 1. We append executable code generation samples $\mathcal{C}_{\text{sample}}$ to the previous in-context examples $\mathcal{E}_{\mathcal{P}}$ and obtain few-shot code generation demonstrations $\mathcal{E}_{\mathcal{C}} = [\mathcal{E}_{\mathcal{P}}; \mathcal{C}_{\text{sample}}]$

$$\begin{aligned} \mathcal{C}_i &\sim p_{\mathcal{M}}(T_i^{\mathcal{C}} \mid T_{:i-1}^{\mathcal{C}} \mathcal{E}_{\mathcal{C}}, \mathcal{Q}, \mathcal{I}^{\mathcal{P}}, \mathcal{P}, \mathcal{I}^{\mathcal{C}}) \\ \mathcal{F}_{code}, \mathcal{R}_{code}, \mathcal{G}_{code} &= \text{EXTRACT}(\mathcal{C}_i), \end{aligned} \tag{2}$$

where $\mathcal{C}_i$ and $T_i^{\mathcal{C}}$ is the i-th token in the generated *code* $\mathcal{C}$. We detail the benefits of Prolog and the reasoning behind our choice in section A.5.

Simulating Search After generating the logic-programming *code*, we want to simulate program execution by generating a problem space traversal trace with our LLM $\mathcal{M}$. We use instructions $\mathcal{I}^{\mathcal{S}}$ and update our in-context samples by appending search traces $\mathcal{S}_{\text{sample}}$ constructed from Prolog execution of sample codes $\mathcal{C}_{\text{sample}}$, i.e. $\mathcal{E}_{\mathcal{S}} = [\mathcal{E}_{\mathcal{C}}; \mathcal{S}_{\text{sample}}]$:

$$\mathcal{S}_i \sim p_{\mathcal{M}}(T_i^{\mathcal{S}} \mid T_{:i-1}^{\mathcal{S}} \mathcal{E}_{\mathcal{C}}, \mathcal{Q}, \mathcal{I}^{\mathcal{P}}, \mathcal{P}, \mathcal{I}^{\mathcal{C}}, \mathcal{C}, \mathcal{I}^{\mathcal{S}}) \tag{3}$$

$$\mathcal{A}_{search}, \mathcal{F}_{search}, \mathcal{R}_{search} = \text{EXTRACT}(\mathcal{S}_i),$$

where $T_i^{\mathcal{S}}$ is the i-th token in the generated *search* trace $\mathcal{S}$. During iterative problem space traversal, we can segment the facts $\mathcal{F}_{search}$, relations $\mathcal{R}_{search}$, completed and backtracked paths with their answers $\mathcal{A}_{search}$ used during the search simulation. To get the final answer we update in-context samples with their correct final answers $\mathcal{A}_{\text{sample}}$ from the executed search $\mathcal{S}_{\text{sample}}$, $\mathcal{E}_{\mathcal{A}} = [\mathcal{E}_{\mathcal{S}}; \mathcal{A}_{\text{sample}}]$ and use instructions $\mathcal{I}^{\mathcal{A}}$ to obtain the final answer from the model.

$$\begin{aligned} \mathcal{A}_{Final} \sim p_{\mathcal{M}}(T_i^{\mathcal{A}} \mid T_{:i-1}^{\mathcal{A}} \mathcal{E}_{\mathcal{C}}, \mathcal{Q}, \\ \mathcal{I}^{\mathcal{P}}, \mathcal{P}, \mathcal{I}^{\mathcal{C}}, \mathcal{C}, \mathcal{I}^{\mathcal{S}}, \mathcal{S}, \mathcal{I}^{\mathcal{A}}) \end{aligned} \tag{4}$$

The prompts used for generating each part in FLARE can be seen in section A.1 along with a complete example in table 9 and a pseudo-code in section A.5.

3.2 Detecting Reasoning Inconsistencies

For each query $\mathcal{Q}$ given the *code* $\mathcal{C}$ and the simulated *search* $\mathcal{S}$ along with the extracted facts $\mathcal{F}_{code}, \mathcal{F}_{search}$ and relations $\mathcal{R}_{code}, \mathcal{R}_{search}$ from each designated module, we aim to detect the inconsistencies during the reasoning process of the LLM. We use exact string matching between all these facts and relations in *code* and simulated *search*.

$$\begin{aligned} &\forall i, \exists j \quad \text{such that} \quad \mathcal{F}^i_{\text{code}} = \mathcal{F}^j_{\text{search}} \\ &\quad \text{and} \quad \forall v, \exists q \quad \mathcal{R}^v_{\text{code}} = \mathcal{R}^q_{\text{search}} \end{aligned} \tag{5}$$

$$\begin{aligned} &\forall j, \exists i \quad \text{such that} \quad \mathcal{F}^i_{\text{code}} = \mathcal{F}^j_{\text{search}} \\ &\quad \text{and} \quad \forall q, \exists v \quad \mathcal{R}^v_{\text{code}} = \mathcal{R}^q_{\text{search}} \end{aligned} \tag{6}$$

With this framework in mind, we define two reasoning failure modes. In the *first* failure mode, given that some fact or relation was used in the simulated *search* but did not exist in the generated *code*, i.e. $\exists j$ such that $\mathcal{F}^j_{\text{search}} \notin \mathcal{F}_{\text{code}}$, we claim that the LLM has *hallucinated*. We postulate that the model either produced incomplete knowledge

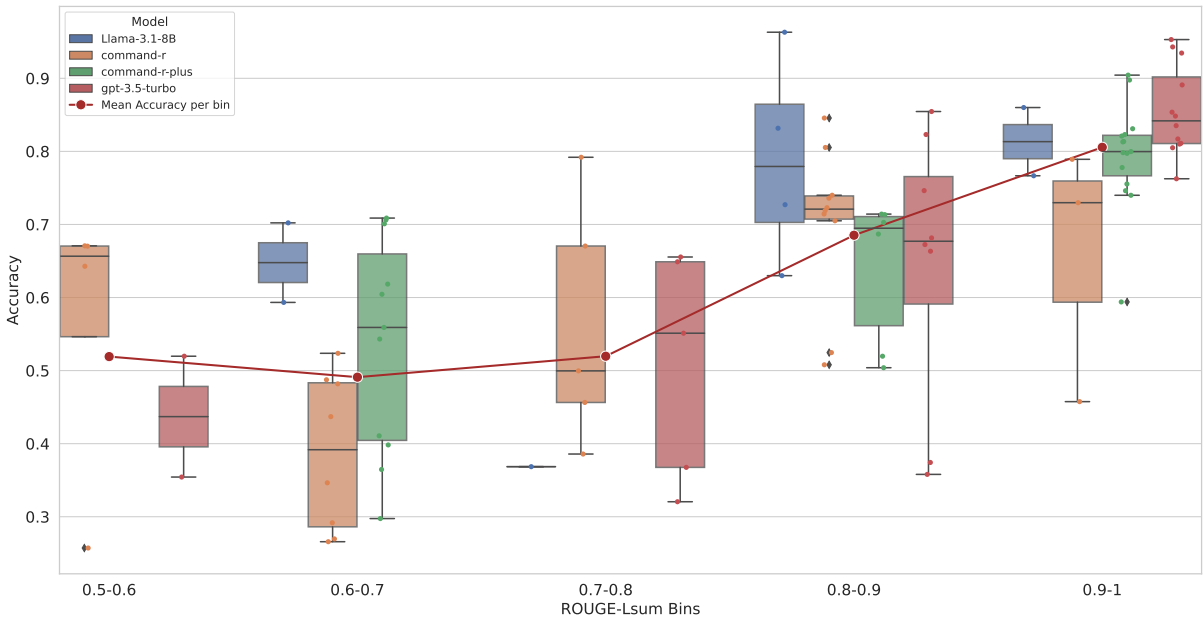

Figure 2: The trend of mean model accuracy w.r.t mean faithfulness for all the models.

Dataset	ChatGPT (gpt-3.5-turbo)					GPT-4 (gpt-4o)				
	Standard	CoT	Logic-LM	FLARE	FLARE$_{SR=2}$	Standard	CoT	Logic-LM	FLARE	FLARE$_{SR=2}$
PrOntoQA	47.40	67.80	61.00	73.40	**79.40**	77.40	98.79	83.20	98.87	**99.24**
LogicalDeduction	40.00	42.33	**65.67**	58.60	64.43	71.33	75.25	87.63	88.00	**90.33**
AR-LSAT	20.34	17.31	26.41	27.39	**30.73**	33.33	35.06	43.04	39.82	**45.02**

Table 2: Comparison of results across datasets for ChatGPT (gpt-3.5-turbo) and GPT-4 (gpt-4o) using Standard, CoT, Logic-LM, FLARE, and FLARE$_{SR=2}$ approaches. *SR=2* refers to a maximum of 2 iterations of code self-refinement.

during formalisation to *code* or created a piece of non-existing information during the *search*. We do not consider facts that emerged during a direct inference step within the simulated search during our calculation. For example, if we are dealing with a mathematical query $4 \cdot (5 + 6) = ?$, the search would involve separately evaluating the expression $5 + 6 = 11$. In this case, 11 will not be treated as a hallucinated fact within the search but rather as an emergent fact obtained from direct inference. The *second* failure mode is the reciprocal case, where a fact or relation present in the *code* is not used during the *search*. We refer to this phenomenon as *sub-optimal reasoning* as it shows that the LLM could not explore the problem space completely or injected unsuitable knowledge during formalisation into *code*.

3.3 Measuring Faithfulness

We propose a method to measure the faithfulness of the LLM reasoning process when using FLARE. As mentioned in section 3.1, for each query in a dataset $\mathcal{D} = [\mathcal{Q}_1, \ldots, \mathcal{Q}_{|\mathcal{D}|}]$, we generate a set of codes $\Phi = [\mathcal{C}_1, \ldots, \mathcal{C}_{|\Phi|}]$ and simulated problem space searches $\Psi = [\mathcal{S}_1, \ldots, \mathcal{S}_{|\Psi|}]$. We use the Prolog engine to execute all of the codes Φ and obtain a set of correctly written programs Φ' and exact search paths Ψ'. As we do not require explicit programmatic correctness during inference in FLARE for any code $\mathcal{C}_i$, some Prolog executions resulting in an error are filtered out in Ψ'. To assess model reasoning faithfulness towards code formalisations, we compare the search paths Φ' obtained from Prolog execution with their designated counterparts Φ'_{gen} generated by the LLM from the same code. We use ROUGE (Lin, 2004) to compute the matching score for each executed and simulated search path. In particular, we use ROUGE-Lsum, which uses the longest common subsequence (LCS) over each line to obtain the final score. This method fits our cause as a line in a Prolog search execution represents a single logic step within the traversal. This allows us to measure the similarity of the reasoning contents and structure in exact and simulated

searches. We have also used other string-matching techniques, all of which show the same trends; thus, we report our results with ROUGE-Lsum.

4 Experimental Setup

Datasets To evaluate FLARE, we use a benchmark of 9 tasks spanning Math Word Problems (MWP), multi-hop QA, relation inference, and 3 logical reasoning datasets. For numerical and mathematical reasoning, we follow CoT (Wei et al., 2022) and include GSM8K (Cobbe et al., 2021), SVAMP (Patel et al., 2021), MultiArith (Roy and Roth, 2015), ASDiv (Miao et al., 2020), and AQuA (Ling et al., 2017). GSM8K, SVAMP, MultiArith, and ASDiv focus on elementary and middle school arithmetic with integer or decimal answers. AQuA involves multiple-choice symbolic reasoning with expressions not explicitly defined in the query. We also test FLARE on three multi-hop QA tasks. StrategyQA (Geva et al., 2021) requires boolean reasoning with sub-goal decomposition (e.g., *"Do all parts of the aloe vera plant taste good?"* in fig. 1). We further use the Date and Sports Understanding subsets from BIG-Bench (bench authors, 2023), which involve temporal and feasibility-based reasoning. For relation inference, we use CLUTRR (Sinha et al., 2019), which requires deducing familial relations from partial graph descriptions in natural language. We evaluate logical reasoning using ProntoQA (Saparov and He, 2023), AR-LSAT (Zhong et al., 2021), and LogicalDeductions from BIG-Bench (et al., 2023), focusing on the challenging subsets of (Pan et al., 2023). These cover deductive, analytical, and logical tasks. Dataset details and examples are in table 7 of section A.1. We also study how model size affects performance and faithfulness (section A.1).

Benchmarks We compare FLARE with CoT (Wei et al., 2022), which uses natural language reasoning chains, and with F-CoT (Lyu et al., 2023) and Logic-LM (Pan et al., 2023), which formalise queries into code and delegate reasoning to external solvers. Evaluated models include Llama3.1 (8B)(Dubey et al., 2024), CmDR (30B) and CmDR+ (100B)(Cohere, 2024), and GPT-3.5 (Brown et al., 2020b) ($\geq$100B (Ye et al., 2023)). As OpenAI Codex (code-DaVinci-002) (Chen et al., 2021) used in F-CoT has been deprecated, we replace it with the new GPT3.5 as suggested by OpenAI and recalculate the results.

5 Results

5.1 Few-shot prompting

To evaluate FLARE, we use a set of models of varying sizes on diverse benchmarks, as defined in section 4. We compare the performance of each model while using FLARE, CoT and F-CoT prompting. The results for F-CoT and CoT on all the models are computed using the codebase of the original study (Lyu et al., 2023). We additionally compare Logic-LM and FLARE using the logic reasoning benchmarks proposed in (Pan et al., 2023).

LLMs for general reasoning Our results, presented in table 1, show that using FLARE allows the LLMs to achieve state-of-the-art results on 7 out of 9 datasets, with an average 28% increase over CoT. We can see a clear trend that FLARE increases the performance compared to CoT and F-CoT for all the models of varying scales. We also see that LLMs not explicitly tuned for coding suffer massive degeneracies when using F-CoT. We postulate that they cannot consistently produce executable programs that satisfy a predefined scheme in F-CoT, thus resulting in an error during execution. This further highlights the value of simulating program execution using an LLM instead of external solvers. The results show that using FLARE yields more benefit on datasets that require longer chains of multi-hop and symbolic reasoning, like AQuA and StrategyQA. Our findings in table 2 show that FLAREachieves state-of-the-art results on 2 out of 3 logic inference benchmarks with 10% increase over CoT and 7% increase over Logic-LM. Following the practice in (Pan et al., 2023, Logic-LM), we also add 2 iterations of code self-refinement to FLARE and show that the model model is able to achieve SOTA results on all 3 benchmarks.

LLMs for code generation To understand the effect of FLARE on models tuned for coding, we use GPT3.5 (Brown et al., 2020a) as it was the OpenAI suggested succession model for Codex (Chen et al., 2021) which is used in F-CoT and possesses strong coding capabilities (Ye et al., 2023). The results in table 1 show that using FLARE is beneficial for models tuned for coding and boosts accuracy with a 16% increase over F-CoT and 9% over CoT. The reason is that many natural language queries with non-trivial formalisations are more suited to be tackled with more commonsense soft reasoning than direct code execution. This is evident in table 1

Method	$\text{CmDR}_{\text{plan-only}}$	$\text{CmDR}_{\text{FLARE}}$	$\text{CmDR+}_{\text{plan-only}}$	$\text{CmDR+}_{\text{FLARE}}$	$\text{GPT-3.5}_{\text{plan-only}}$	$\text{GPT-3.5}_{\text{FLARE}}$
GSM8K	24.7	**52.4**	40.7	**71.4**	36.1	**68.1**
AQuA	35.0	**43.7**	55.1	**55.9**	54.3	**55.1**
StrategyQA	65.5	**67.0**	**75.7**	70.8	62.3	**65.5**

Table 3: The table shows the accuracy of an LLM with FLARE compared to prompting for a final answer directly after generating (plan-only) a plan $\mathcal{P}$.

Model	#Paths	#Hops/p	#Fails/p	TotHops	TotFails
Incorrect Answers					
$\text{Llama-3.1-8B}_{\text{FLARE}}$	1.55	11.12	1.52	15.09	2.26
$\text{CmDR}_{\text{FLARE}}$	1.51	6.55	0.68	10.56	1.39
$\text{CmDR+}_{\text{FLARE}}$	0.92	7.52	1.13	8.57	1.32
GPT-3.5	0.68	5.22	0.71	5.32	0.74
Correct Answers					
$\text{Llama-3.1-8B}_{\text{FLARE}}$	1.43	9.12	0.62	12.36	0.96
$\text{CmDR}_{\text{FLARE}}$	1.19	7.10	0.42	11.29	0.66
$\text{CmDR+}_{\text{FLARE}}$	0.97	7.19	0.42	8.22	0.61
$\text{GPT-3.5}_{\text{FLARE}}$	0.82	5.65	0.26	5.69	0.27

Table 4: **#Paths**: Avg. number of reasoning paths tried by the model. **#Hops/p**: Avg. number of hops per path. **#Fails/p**: Avg. number of fails (unsuccessful hops) per path. **TotHops**: Avg. total hops (summed across all paths). **TotFails**: Avg. total fails (summed across all paths). The purple cells show that incorrect reasoning paths often have fewer failed search paths.

Model	UEF (%) in Search	OR (%)	UR (%) in code
Correct Answers			
$\text{Llama-3.1-8B}_{\text{FLARE}}$	74.14	43.65	5.73
$\text{CmDR}_{\text{FLARE}}$	59.06	35.96	4.02
$\text{CmDR+}_{\text{FLARE}}$	64.30	34.47	4.54
$\text{GPT-3.5}_{\text{FLARE}}$	64.46	37.55	1.90
Avg. (Correct)	65.49	37.91	4.05
Incorrect Answers			
$\text{Llama-3.1-8B}_{\text{FLARE}}$	54.69	35.04	9.28
$\text{CmDR}_{\text{FLARE}}$	54.50	32.76	6.23
$\text{CmDR+}_{\text{FLARE}}$	44.12	24.98	8.22
$\text{GPT-3.5}_{\text{FLARE}}$	36.02	24.44	6.94
Avg. (Incorrect)	47.33	29.31	7.67
Δ	18.16	8.60	-3.62

Table 5: The table shows how the percentage of unique emergent facts (UEF) in search, overlapping relations (OR) between code and search, and unused relations (UR) in code impact answer correctness.

where FLARE and CoT are consistently better than F-CoT in StrategyQA, Sports and CLUTRR. The opposite case of numeric and algorithmic heavy reasoning tasks is also covered by FLARE as it maintains strong performance similar to F-CoT on MWP problems table 1. Consequently, FLARE allows combining algorithmic formalisation with simulated soft-reasoning, circumventing the pitfalls of using a deterministic external solver while still producing a query formalisation and problem space traversal.

5.2 Is simulating search useful?

To understand if simulating a search over the problem space is useful, we compare the performance of FLARE where we only generate the *plan* without the subsequent *code* or *search* components. We refer to this framework setup as *plan-only*, which can be seen in fig. 1 if we were to use only the *plan* for answer generation. We completed this ablation using CmDR, CmDR+, and GPT-3.5, and we used GSM8K, AQuA, and StrategyQA as our baselines. The results in table 3 confirm that all of the models suffer massive performance degradation from $61.1 \rightarrow 49.9$ when omitting the *code* and the *search* components of FLARE. We hypothesise that this is caused by insufficient problem space exploration when using the *plan-only* setting. Furthermore, we have already seen in table 1 that in methods, like F-CoT, that do not use simulated problem space exploration for soft-reasoning and only rely on *plan* and *code*, the performance also deteriorates even resulting in a complete breakdown of reasoning over the designated datasets. This can be viewed as a constrained version of FLARE with *code-only* execution. Consequently, our results show that simulating problem space traversal is highly beneficial as it avoids the pitfalls posed by *plan-only* and *code-only* modes by exploring the problem space more rigorously and soft-reasoning during that traversal instead of using external solvers.

5.3 Faithful Reasoning Improves Performance

As described in section 3, using FLARE allows us to measure the faithfulness of the LLM reasoning process by comparing the simulated problem space traversals Φ'_{gen} with actual traces Φ' produced from a symbolic Prolog solver. To do this, we initially compute the percentage of syntactically correct executable code each LLM produces. We have observed that all of the models are capable of producing correct executable Prolog code in 67% of cases

on average and $\geq 50\%$ of cases at the very least. The complete details can be seen in the top part of fig. 3 in section A.5. This shows that the simulated searches Φ'_{gen} can be considered a representative sample that will be further used to accurately measure the faithfulness of the simulated search w.r.t. the generated code. After measuring the reasoning faithfulness for each model, we want to understand what impact it has on the performance of the LLM. In fig. 2, we segment the models w.r.t. their ROUGE-Lsum scores. The results show that model performance is strongly positively correlated with reasoning faithfulness. However, we also observe that executing semantically precise code results in an accurate answer only in 47% of cases on average. Refer to the bottom part of fig. 3 in section A.5 for more details. Indeed, having a simulated search trace with a ROUGE-Lsum faithfulness score of 1, would be equivalent to simply executing the program as proposed in F-CoT. Yet we have priorly shown that F-CoT struggles with reasoning tasks that are hard to formalise and require multi-hop commonsense and soft reasoning. These two discoveries show that optimal LLM reasoning, conditioned on a search in the problem space, should be increasingly faithful toward the facts, relations and the search strategy defined within the code while simultaneously maintaining the capability for soft-reasoning along more abstractly defined concepts. Our results show that FLARE allows LLMs to maintain a similar reasoning capacity.

5.4 What is important during the search?

We also analyze the reasons which can lead to optimal reasoning within an LLM. We calculate several statistics, like the average number of explored paths and the average and total hops and failures per path, for each model during the simulated traversal. The failure in a path is an invalidation of a solution for a sub-goal explored during the search, which is used for backtracking, as explained in section 3. Calculating these statistics is simple as the *search* component of FLARE, seen in fig. 1, is a structured simulation of a Prolog trace, where each line contains a hop of reasoning inference. We split these statistics for the reasoning paths that lead to correct or incorrect outcomes. Our results in table 4 show that LLM performance and reasoning optimality are not directly connected to the amount of explored paths or multi-hop inferences per path. We also see that traces that lead to incorrect answers have a higher number of failures per path and in total. We hypothesise that LLMs with traces that were optimal for reasoning and led to correct answers could skip exploring degenerate solutions due to strong commonsense reasoning capabilities. Further analyses focus on identifying inconsistencies and failure modes (section 3.2). By comparing relations in code with those in search traces, we measure emergent hallucinations and unused relations, highlighting areas of sub-optimal reasoning. We also assess the uniqueness of emergent facts per inference hop, indicating the extent of problem-space exploration (table 5). The results in table 5 show consistently over each model that, on average, traces that lead to correct answers had a higher percentage of unique emergent facts (UEF) and overlap in the relations (OR) used between the code and search, while the portion of underutilized relations (UR) was lower. This means that optimal reasoning with an LLM requires a great degree of problem-space exploration with fewer relation hallucinations during the search and more relation utilization from the defined code. This aligns with our prior discoveries, which show a strong correlation between simulated search faithfulness towards the formalised code and model performance.

6 Conclusion

This work introduces FLARE, a novel approach for logic-aided interpretable formalisation and reasoning with simulated search over the problem space. We show that models of varying scales obtain state-of-the-art results compared to prompting paradigms like CoT, F-CoT and Logic-LM. We further pinpoint that using FLARE allows us to perform soft reasoning with simulated search, making it flexible for diverse reasoning benchmarks. We introduce a method to measure model reasoning faithfulness w.r.t. the problem formalization ingrained within FLARE. Our results show that model performance is positively correlated with the faithfulness of the reasoning process. The systematic studies of the method show the benefits of using simulated search compared to natural language reasoning and external symbolic solvers. We further show that using FLARE allows us to interpretably and rigorously detect hallucinations and sub-optimal and inconsistent reasoning patterns.

Limitations

While FLARE offers significant improvements in faithfulness and interoperability, it depends on the

quality of LLM-generated plans and code; errors or omissions in formalisation can propagate through the simulated search, potentially leading to incorrect or incomplete answers. The generation of formal code and simulated search traces depends heavily on the LLM's prompt-following ability. The simulation of code execution may not fully explore extremely large or open-ended problem spaces, and prompt sensitivity can affect search thoroughness.

Risks and Impact Statement

FLARE advances the capabilities of large language models in logical reasoning and problem-solving, with potential positive impacts on applications requiring transparent and verifiable decision-making processes. The ability of the method to formalise reasoning steps and detect inconsistencies could improve reliability in high-stakes domains like healthcare decision support, educational assessment, and automated planning systems. However, this advancement also raises important considerations — the improved reasoning capabilities could be misused to automate deceptive or manipulative argumentation, and the increased persuasiveness through logical formalisation could mask underlying biases or false premises. Additionally, while FLARE improves transparency in reasoning, it may create a false sense of rigour in cases where the underlying logic is flawed but presented in a formally convincing manner.

Acknowledgments

Erik is partially funded by a DFF Sapere Aude research leader grant under grant agreement No 0171-00034B, as well as by an NEC PhD fellowship, and is supported by the Pioneer Centre for AI, DNRF grant number P1. Pasquale was partially funded by ELIAI (The Edinburgh Laboratory for Integrated Artificial Intelligence), EPSRC (grant no. EP/W002876/1), an industry grant from Cisco, and a donation from Accenture LLP. Isabelle's research is partially funded by the European Union (ERC, ExplainYourself, 101077481), and is supported by the Pioneer Centre for AI, DNRF grant number P1. This work was supported by the Edinburgh International Data Facility (EIDF) and the Data-Driven Innovation Programme at the University of Edinburgh.

References

Viraat Aryabumi, Yixuan Su, Raymond Ma, Adrien Morisot, Ivan Zhang, Acyr Locatelli, Marzieh Fadaee, Ahmet Üstün, and Sara Hooker. 2024. To code, or not to code? exploring impact of code in pre-training. *CoRR*, abs/2408.10914.

Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. 2021. Program synthesis with large language models. *CoRR*, abs/2108.07732.

BIG bench authors. 2023. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research*.

Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. 2024. Graph of thoughts: Solving elaborate problems with large language models. In *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2014, February 20-27, 2024, Vancouver, Canada*, pages 17682–17690. AAAI Press.

Kenneth A. Bowen. 1979. Prolog. In *Proceedings of the 1979 Annual Conference, Detroit, Michigan, USA, October 29-31, 1979*, pages 14–23. ACM.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, and 12 others. 2020a. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, and 12 others. 2020b. Language models are few-shot learners. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*.

Hyungjoo Chae, Yeonghyeon Kim, Seungone Kim, Kai Tzu-iunn Ong, Beong-woo Kwak, Moohyeon Kim, Seonghwan Kim, Taeyoon Kwon, Jiwan Chung, Youngjae Yu, and Jinyoung Yeo. 2024. Language models as compilers: Simulating pseudocode execution improves algorithmic reasoning in language models. *CoRR*, abs/2404.02575.

Ashok K. Chandra and David Harel. 1985. Horn clauses queries and generalizations. *J. Log. Program.*, 2(1):1–15.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, and 39 others. 2021. Evaluating large language models trained on code. *CoRR*, abs/2107.03374.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. 2023. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Trans. Mach. Learn. Res.*, 2023.

Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, Dian Yu, Linfeng Song, Qiuzhi Liu, Mengfei Zhou, Zhuosheng Zhang, and 1 others. 2024. Do not think that much for 2+ 3=? on the overthinking of o1-like llms. *arXiv preprint arXiv:2412.21187*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *CoRR*, abs/2110.14168.

Cohere. 2024. Command r: Retrieval-augmented generation at production scale. `https://txt.cohere.com/command-r`.

Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, and 82 others. 2024. The llama 3 herd of models. *CoRR*, abs/2407.21783.

Aarohi Srivastava et al. 2023. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Trans. Mach. Learn. Res.*, 2023.

Kanishk Gandhi, Denise Lee, Gabriel Grand, Muxin Liu, Winson Cheng, Archit Sharma, and Noah D. Goodman. 2024. Stream of search (sos): Learning to search in language. *CoRR*, abs/2404.03683.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: program-aided language models. In *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, volume 202 of *Proceedings of Machine Learning Research*, pages 10764–10799. PMLR.

Mor Geva, Daniel Khashabi, Elad Segal, Tushar Khot, Dan Roth, and Jonathan Berant. 2021. Did aristotle use a laptop? A question answering benchmark with implicit reasoning strategies. *Trans. Assoc. Comput. Linguistics*, 9:346–361.

Leilani H. Gilpin, David Bau, Ben Z. Yuan, Ayesha Bajwa, Michael A. Specter, and Lalana Kagal. 2018. Explaining explanations: An overview of interpretability of machine learning. In *5th IEEE International Conference on Data Science and Advanced Analytics, DSAA 2018, Turin, Italy, October 1-3, 2018*, pages 80–89. IEEE.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021a. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021b. Measuring mathematical problem solving with the MATH dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*.

Bernease Herman. 2017. The promise and peril of human evaluation for model interpretability. *arXiv preprint arXiv:1711.07414*.

Alon Jacovi, Yonatan Bitton, Bernd Bohnet, Jonathan Herzig, Or Honovich, Michael Tseng, Michael Collins, Roee Aharoni, and Mor Geva. 2024. A chain-of-thought is as strong as its weakest link: A benchmark for verifiers of reasoning chains. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, pages 4615–4634. Association for Computational Linguistics.

Alon Jacovi and Yoav Goldberg. 2020. Towards faithfully interpretable NLP systems: How should we define and evaluate faithfulness? In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*, pages 4198–4205. Association for Computational Linguistics.

Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A survey on large language models for code generation. *CoRR*, abs/2406.00515.

Zhanming Jie, Trung Quoc Luong, Xinbo Zhang, Xiaoran Jin, and Hang Li. 2023. Design of chain-of-thought in math problem solving. *arXiv preprint arXiv:2309.11054*.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*.

Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, Kamile Lukosiute, Karina Nguyen, Newton Cheng, Nicholas Joseph, Nicholas Schiefer, Oliver Rausch, Robin Larson, Sam McCandlish, Sandipan Kundu, and 11 others. 2023a. Measuring faithfulness in chain-of-thought reasoning. *CoRR*, abs/2307.13702.

Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, and 1 others. 2023b. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint arXiv:2307.13702*.

Lucas Lehnert, Sainbayar Sukhbaatar, Paul McVay, Michael Rabbat, and Yuandong Tian. 2024. Beyond a*: Better planning with transformers via search dynamics bootstrapping. *CoRR*, abs/2402.14083.

Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay V. Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. 2022. Solving quantitative reasoning problems with language models. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*.

Chengshu Li, Jacky Liang, Andy Zeng, Xinyun Chen, Karol Hausman, Dorsa Sadigh, Sergey Levine, Li Fei-Fei, Fei Xia, and Brian Ichter. 2024. Chain of code: Reasoning with a language model-augmented code emulator. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net.

Chin-Yew Lin. 2004. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain. Association for Computational Linguistics.

Wang Ling, Dani Yogatama, Chris Dyer, and Phil Blunsom. 2017. Program induction by rationale generation: Learning to solve and explain algebraic word problems. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, ACL 2017, Vancouver, Canada, July 30 - August 4, Volume 1: Long Papers*, pages 158–167. Association for Computational Linguistics.

Zhan Ling, Yunhao Fang, Xuanlin Li, Zhiao Huang, Mingu Lee, Roland Memisevic, and Hao Su. 2023. Deductive verification of chain-of-thought reasoning. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. 2023. LLM+P: empowering large language models with optimal planning proficiency. *CoRR*, abs/2304.11477.

Fang Liu, Yang Liu, Lin Shi, Houkun Huang, Ruifeng Wang, Zhen Yang, and Li Zhang. 2024. Exploring and evaluating hallucinations in llm-powered code generation. *CoRR*, abs/2404.00971.

John W. Lloyd. 1994. Practical advtanages of declarative programming. In *1994 Joint Conference on Declarative Programming, GULP-PRODE'94 Peñiscola, Spain, September 19-22, 1994, Volume 1*, pages 18–30.

Qing Lyu, Shreya Havaldar, Adam Stein, Li Zhang, Delip Rao, Eric Wong, Marianna Apidianaki, and Chris Callison-Burch. 2023. Faithful chain-of-thought reasoning. In *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics, IJCNLP 2023 -Volume 1: Long Papers, Nusa Dua, Bali, November 1 - 4, 2023*, pages 305–329. Association for Computational Linguistics.

Aman Madaan and Amir Yazdanbakhsh. 2022a. Text and patterns: For effective chain of thought, it takes two to tango. *arXiv preprint arXiv:2209.07686*.

Aman Madaan and Amir Yazdanbakhsh. 2022b. Text and patterns: For effective chain of thought, it takes two to tango. *CoRR*, abs/2209.07686.

Aman Madaan, Shuyan Zhou, Uri Alon, Yiming Yang, and Graham Neubig. 2022. Language models of code are few-shot commonsense learners. *arXiv preprint arXiv:2210.07128*.

Shen-Yun Miao, Chao-Chun Liang, and Keh-Yih Su. 2020. A diverse corpus for evaluating and developing english math word problem solvers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*, pages 975–984. Association for Computational Linguistics.

Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. Codegen: An open large language model for code with multi-turn program synthesis. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.

Liangming Pan, Alon Albalak, Xinyi Wang, and William Yang Wang. 2023. Logic-lm: Empowering large language models with symbolic solvers for faithful logical reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2023, Singapore, December 6-10, 2023*, pages 3806–3824. Association for Computational Linguistics.

Arkil Patel, Satwik Bhattamishra, and Navin Goyal. 2021. Are NLP models really able to solve simple math word problems? In *Proceedings of the 2021 Conference of the North American Chapter of the*

Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021, pages 2080–2094. Association for Computational Linguistics.

Subhro Roy and Dan Roth. 2015. Solving general arithmetic word problems. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing, EMNLP 2015, Lisbon, Portugal, September 17-21, 2015*, pages 1743–1752. The Association for Computational Linguistics.

Abulhair Saparov and He He. 2023. Language models are greedy reasoners: A systematic formal analysis of chain-of-thought. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.

Koustuv Sinha, Shagun Sodhani, Jin Dong, Joelle Pineau, and William L. Hamilton. 2019. CLUTRR: A diagnostic benchmark for inductive reasoning from text. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019, Hong Kong, China, November 3-7, 2019*, pages 4505–4514. Association for Computational Linguistics.

Zayne Sprague, Fangcong Yin, Juan Diego Rodriguez, Dongwei Jiang, Manya Wadhwa, Prasann Singhal, Xinyu Zhao, Xi Ye, Kyle Mahowald, and Greg Durrett. 2024. To cot or not to cot? chain-of-thought helps mainly on math and symbolic reasoning. *arXiv preprint arXiv:2409.12183*.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc Le, Ed Chi, Denny Zhou, and Jason Wei. 2023. Challenging BIG-bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 13003–13051, Toronto, Canada. Association for Computational Linguistics.

Christian Szegedy. 2020. A promising path towards autoformalization and general artificial intelligence. In *Intelligent Computer Mathematics - 13th International Conference, CICM 2020, Bertinoro, Italy, July 26-31, 2020, Proceedings*, volume 12236 of *Lecture Notes in Computer Science*, pages 3–20. Springer.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. 2023. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

Karthik Valmeekam, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. 2022. Large language models still can't plan (a benchmark for llms on planning and reasoning about change). In *NeurIPS 2022 Foundation Models for Decision Making Workshop*.

Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. 2023a. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9-14, 2023*, pages 2609–2634. Association for Computational Linguistics.

Qingxiang Wang, Cezary Kaliszyk, and Josef Urban. 2018. First experiments with neural translation of informal to formal mathematics. In *Intelligent Computer Mathematics - 11th International Conference, CICM 2018, Hagenberg, Austria, August 13-17, 2018, Proceedings*, volume 11006 of *Lecture Notes in Computer Science*, pages 255–270. Springer.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023b. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.

Yue Wang, Qiuzhi Liu, Jiahao Xu, Tian Liang, Xingyu Chen, Zhiwei He, Linfeng Song, Dian Yu, Juntao Li, Zhuosheng Zhang, and 1 others. 2025. Thoughts are all over the place: On the underthinking of o1-like llms. *arXiv preprint arXiv:2501.18585*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, and 1 others. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.

Jan Wielemaker, Tom Schrijvers, Markus Triska, and Torbjörn Lager. 2012. Swi-prolog. *Theory and Practice of Logic Programming*, 12(1-2):67–96.

Wikipedia. 2024. Countdown (game show) — Wikipedia, the free encyclopedia. http://en.wikipedia.org/w/index.php?title=Countdown%20(game%20show)&oldid=1248084922. [Online; accessed 09-September-2024].

Yuhuai Wu, Albert Qiaochu Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. 2022. Autoformalization with large language models. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*.

Mengjiao Yang, Dale Schuurmans, Pieter Abbeel, and Ofir Nachum. 2022. Chain of thought imitation with procedure cloning. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023a. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023b. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.

Junjie Ye, Xuanting Chen, Nuo Xu, Can Zu, Zekai Shao, Shichun Liu, Yuhan Cui, Zeyang Zhou, Chao Gong, Yang Shen, Jie Zhou, Siming Chen, Tao Gui, Qi Zhang, and Xuanjing Huang. 2023. A comprehensive capability analysis of GPT-3 and GPT-3.5 series models. *CoRR*, abs/2303.10420.

Wenyuan Zhang, Shuaiyi Nie, Xinghua Zhang, Zefeng Zhang, and Tingwen Liu. 2025. S1-bench: A simple benchmark for evaluating system 1 thinking capability of large reasoning models. *arXiv preprint arXiv:2504.10368*.

Yadong Zhang, Shaoguang Mao, Tao Ge, Xun Wang, Adrian de Wynter, Yan Xia, Wenshan Wu, Ting Song, Man Lan, and Furu Wei. 2024. LLM as a mastermind: A survey of strategic reasoning with large language models. *CoRR*, abs/2404.01230.

Wanjun Zhong, Siyuan Wang, Duyu Tang, Zenan Xu, Daya Guo, Jiahai Wang, Jian Yin, Ming Zhou, and Nan Duan. 2021. AR-LSAT: investigating analytical reasoning of text. *CoRR*, abs/2104.06598.

Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V. Le, and Ed H. Chi. 2023. Least-to-most prompting enables complex reasoning in large language models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.

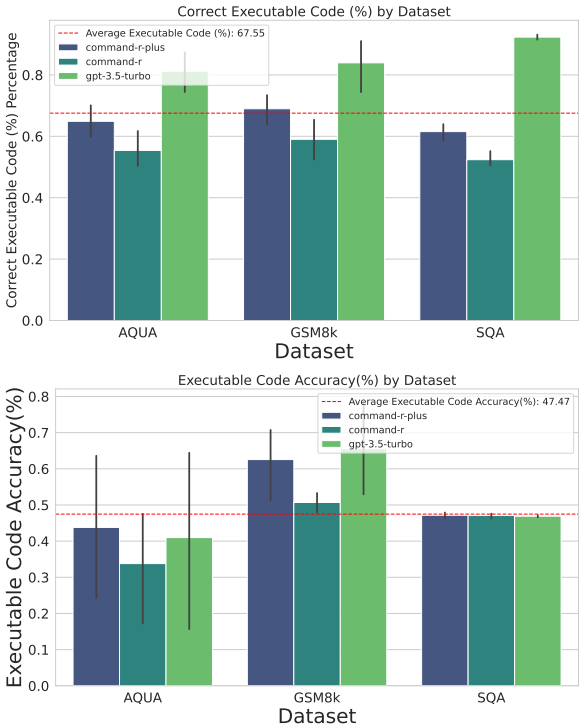

Figure 3: The figure shows the percentage of executable code per model (top) and the accuracy of the executable code when answering the queries (bottom).

Model	Avg. hops per path	Hal. (%)	UK. (%)
Llama-3.1-8B	9.4	63.3	62.9
CmDR	6.7	54.7	56.9
CmDR+	7.2	54.3	56.3
GPT-3.5	5.5	49.3	52.1

Table 6: Changes in simulated search statistics when using FLARE across model scales (8B to 100B+). *Hallucinations* (Hal.) are facts/predicates used only in the trace, while *unutilised knowledge* (UK.) denotes facts/relations appearing only in the code.

A Appendix

A.1 The effect of scale

We want to assess the impact of the number of parameters in the model on the overall performance and faithfulness. The results in fig. 4 show no precise relation between model scale, performance and faithfulness. However, scaled models from the same family, i.e. CmDR (30B) and CmDR+ (100B), show improvements in reasoning faithfulness and model performance. We can also see in table 6 that as the model size increases, the average number of hops and the portion of hallucinations and unutilised knowledge decreases. This further confirms our prior assumptions that models with strong commonsense soft-reasoning capabilities can skip steps during the search while maintaining the knowledge and structure of the traversal strategy outlined in the code.

A.2 LLM Prompts

We define straight-forward prompts for generating *plan*, *code* and *search* simulation in FLARE, which can be observed in section A.5.

A.3 Dataset Statistics

The datasets used in this study encompass a variety of domains, specifically targeting the performance of the models in interpreting Math Word Problems, multi-hop question answering, and relational inference. Table 7 provides a detailed breakdown of each dataset, including the number of few-shot in-context samples (shots), the number of test samples, and representative examples from each dataset. The datasets provide a comprehensive basis for evaluating the models' abilities to handle complex tasks across different domains, facilitating an in-depth analysis of model performance under few-shot conditions.

A.4 FLARE Pseudo-code

Below, we present the pseudo-code for the execution of the *plan*, *code*, and *search* procedures in FLARE. The pseudo-code describes the modular pipeline in FLARE for tackling natural language queries with faithful simulated search. (i) **Plan Generation:** This stage creates a structured natural language outline of the reasoning process, breaking down the query into logical steps and analysis. The plan serves as the foundation for formalization into a logic-based representation. (ii) **Code Generation:** Based on the generated plan, a logic programming code (e.g., in Prolog) is synthesized. This code formalizes the query into a set of facts, relations, and goals, which collectively define the problem space for reasoning. (iii) **Search Simulation:** The generated code is utilized to simulate a search trace over the problem space. This includes iterative reasoning, backtracking when goals are unmet, and extracting emergent facts or relations during the process. Each of these stages is implemented as a modular component. The generation from each of the stages feeds into the next, allowing seamless integration and incremental improvement in reasoning accuracy. A detailed pseudo-code is provided below in section A.5.

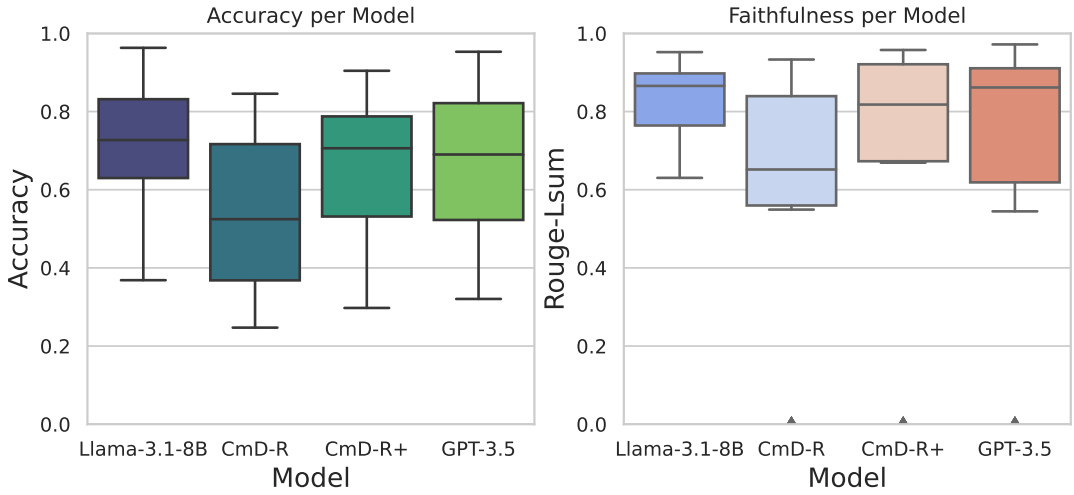

Figure 4: The effect of the model parameter scale from 8B to 100B+ on model accuracy (left) and faithfulness (right).

Domain	Dataset	Shots	Test Samples	Example
Math Word Problems	GSM8K	8	1,319	Q: A robe takes 2 bolts of blue fiber and half that much white fiber. How many bolts in total does it take? A: 3
	SVAMP	8	1,000	Q: Dan had $3 left with him after he bought a candy bar. If he had $4 at the start, how much did the candy bar cost?A: 1
	MultiArith	8	600	Q: A pet store had 13 siamese cats and 5 house cats. During a sale they sold 10 cats. How many cats do they have left? A: 8
	ASDiv	8	2,096	Q: Adam has five more apples than Jackie. Jackie has nine apples. How many apples does Adam have? A: 14
	AQuA	8	254	Q: A man walks at 5 kmph for 6 hrs and at 4 kmph for 12 hrs. His average speed is Answer option: A)4 1/3 km/h, B)7 2/3 km/h, C)9 ½ km/h, D)8 km/h, E)81 km/h A: A
Multi-hop QA	StrategyQA	6	2,290	Q: Did Aristotle use a laptop? A: False
	Date Understanding	10	359	Q: Yesterday was April 30, 2021. What is the date tomorrow in MM/DD/YYYY? A: "05/02/2021"
	Sports Understanding	10	977	Q: Is the following sentence plausible? Lionel Messi was called for icing? A: False
Relational Inference	CLUTRR	8	1,042	Q: [Carlos] is [Clarence]'s brother. [Carlos] and his sister, [Annie], went shopping. asked her mom [Valerie] if she wanted anything, but [Valerie] said no. How is [Valerie] related to [Clarence]? A: "mother"

Table 7: The statistics and examples of the datasets used in benchmarking. Shots refers to the number of few-shot in-context samples used during benchmarking.

A.5 Benefits of Prolog

Prolog is a symbolic logic-programming engine (Bowen, 1979) used for heuristic search over Horn Clauses (Chandra and Harel, 1985). It is a declarative programming paradigm (Lloyd, 1994), meaning that the code is expressed as the logic of computation. In particular, this logic is formalised as a set of facts $\mathcal{F}$ and relations $\mathcal{R}$ forming our problem space, while the final goal $\mathcal{G}$ is a first-order logic combination of them. As a default, Prolog uses a depth-first search (DFS) strategy (Bowen, 1979) for sub-goal decomposition and feasible traversal of the problem space that satisfies the goal $\mathcal{G}$. Such a traversal is referred to as the *trace*. At each trace step, the program can either confirm or invalidate the sub-goal using the feasibility of fact and relation combinations, expand the search tree or retry satisfying a failed sub-goal with new combinations. An example of such a search can be observed in fig. 1. It is possible to complete an exhaustive search, exploring all possible paths that do or do not satisfy the goal. These characteristics are beneficial as we can explicitly access and segment the facts and relations that form the problem space and the search strategy used for query formalisation. As Prolog is declarative, it is sufficient to use a regexp heuristic for the segmentation, which is referred to as EXTRACT in eq. (2) and eq. (3). Furthermore,

including exhaustive traversal traces in-context allows the LLM to simulate sub-goal decomposition, backtracking, intermediate goal invalidation, etc. We discuss this in more depth in the next paragraph.

Algorithm 1 FLARE Methodology: Faithful Logic-Aided Reasoning and Exploration

Require: Query $\mathcal{Q}$, Language Model $\mathcal{M}$
Ensure: Answer $\mathcal{A}$

Initialization: Load few-shot examples for plans ($\mathcal{E}_P$), code ($\mathcal{E}_C$), and search traces ($\mathcal{E}_S$)
Input: Natural language query $\mathcal{Q}$
procedure GENERATE PLAN
    Prompt $\mathcal{M}$ with instructions $\mathcal{I}_P$ and examples $\mathcal{E}_P$ to generate a plan $\mathcal{P}$
    $\mathcal{P} \leftarrow \arg\max\ p_{\mathcal{M}}(T_P|T_{P:<i}, \mathcal{E}_P, \mathcal{Q}, \mathcal{I}_P)$
end procedure
procedure GENERATE CODE
    Append examples $\mathcal{E}_C$ to $\mathcal{E}_P$
    Prompt $\mathcal{M}$ with instructions $\mathcal{I}_C$ to generate logic programming code $\mathcal{C}$
    $\mathcal{C} \leftarrow \arg\max\ p_{\mathcal{M}}(T_C|T_{C:<i}, \mathcal{E}_C, \mathcal{Q}, \mathcal{P}, \mathcal{I}_C)$
    $(F_{\text{code}}, R_{\text{code}}, G_{\text{code}}) \leftarrow \text{EXTRACT}(\mathcal{C})$
end procedure
procedure SIMULATE SEARCH
    Append search trace examples $\mathcal{E}_S$ to $\mathcal{E}_C$
    Prompt $\mathcal{M}$ with instructions $\mathcal{I}_S$ to simulate a search trace $\mathcal{S}$
    $\mathcal{S} \leftarrow \arg\max\ p_{\mathcal{M}}(T_S|T_{S:<i}, \mathcal{E}_S, \mathcal{Q}, \mathcal{P}, \mathcal{C}, \mathcal{I}_S)$
    $(F_{\text{search}}, R_{\text{search}}, \mathcal{A}_{\text{search}}) \leftarrow \text{EXTRACT}(\mathcal{S})$
    while Goal $\mathcal{G}_{\text{code}}$ is not satisfied **do**
        Explore next sub-goal in $\mathcal{S}$
        if Sub-goal fails **then**
            Backtrack to the previous state (Learned through in-context sampels)
        end if
    end while
end procedure
procedure FINAL ANSWER GENERATION
    Append correct answers from $\mathcal{A}_{\text{search}}$ to examples
    Prompt $\mathcal{M}$ with instructions $\mathcal{I}_A$ to finalize answer $\mathcal{A}$
    $\mathcal{A} \leftarrow \arg\max\ p_{\mathcal{M}}(T_A|T_{A:<i}, \mathcal{E}_A, \mathcal{Q}, \mathcal{P}, \mathcal{C}, \mathcal{S}, \mathcal{I}_A)$
end procedure
return $\mathcal{A}$

Task	Prompt	Description
Plan Generation	Generate an explanation and analysis, and plan to generate a prompt for writing a swi-prolog code for the last task. The 3 sections should be exactly outlined. Your plan should show enough intermediate reasoning steps towards the answer. Construct the plan as much as you can and describe the logic specifically. When constructing the plan for the code prompt, actively use swi prolog search capabilities.	Detailed instructions for generating an outline and plan, with an emphasis on reasoning steps and using Prolog's search capabilities.
Code Generation	Write a Prolog code to solve using the plan. If there are unknown or stochastic atoms or predicates, fill in the values for them as a logical assumption and add a comment in the same line Assumed atom/predicate". Do not use write and read commands within the code. The code should be very detailed and utilize swi prolog capabilities to the fullest. To run the program, at the end create a predicate named "query" that returns the correct numerical answer. The last line of the program should be the commented-out driver predicate "query". Write only the code.	Instructions for generating a Prolog code based on the plan with assumptions for unknown atoms. Emphasizes code details and a final "query" predicate.
Simulated Search	Ignoring the read commands, explicitly write out the search paths that are explored by the code: #### Here are the paths [Starting Search Simulation]: #### [Path 1]:	A task to simulate and display the search paths that the Prolog code would follow during execution.
Final Answer	Given the plan, the code and the explored search paths answer the question above. Answer with the correct numerical answer. ##### Here is the answer:	Final prompt asking for the correct numerical answer based on the previous steps.

Table 8: Table of Prompts for Plan, Code, Simulated Search, and Final Answer generation for GSM8K (Cobbe et al., 2021).

Section	Content
Question	"Can Reiki be stored in a bottle?"
Explanation	Reiki is a form of alternative therapy that involves the channeling of energy to promote healing and relaxation. It is often administered through the practitioner's hands, with the energy flowing directly to the recipient. The question raises the possibility of storing Reiki energy in a bottle, which may be a common query due to the abstract nature of energy healing.
Analysis	To address the question of whether Reiki can be stored in a bottle, it is important to consider the nature of Reiki energy, the principles of energy healing, and the practical implications of containing and preserving such energy.
Plan	• Understand Reiki Energy: ... • Explore Energy Storage: ... • Consider Symbolism and Intention: ... • Evaluate Practicality: ... • Reflect on Tradition: ...

Code	`% Define the nature of Reiki energy`
	`reiki_energy(spiritual_energy).`
	`reiki_energy(channeling_through_touch).`
	`% Define the feasibility of storing energy in a bottle`
	`store_energy_in_bottle(feasible) :-`
	`  reiki_energy(spiritual_energy),`
	`  reiki_energy(channeling_through_touch).`
	`% Storing Reiki energy is not a common practice`
	`not_common_practice(store_reiki_in_bottle).`
	`% Query to determine if Reiki can be stored in a bottle`
	`query :-`
	`  store_energy_in_bottle(feasible),`
	`  not_common_practice(store_reiki_in_bottle).`
	`% :- query.`

Search Paths	• 0: Start of execution: Beginning Search • 1: Call: query • 2: Call: store_energy_in_bottle(feasible) • 3: Call: reiki_energy(spiritual_energy) • 4: Call: reiki_energy(channeling_through_touch) • 5: Call: not_common_practice(store_reiki_in_bottle) • 6: Call: feasible=no • 7: Fail: feasible=no | 'Result': 'Search Failed' • 8: Fail: query | 'Result': 'Search Failed'
Answer	"No, Reiki energy cannot be stored in a bottle based on the logical evaluation of its abstract, non-physical nature and traditional practices of Reiki."

Table 9: Complete example of FLARE